

SECURE MESSAGING SYSTEM USING AES ENCRYPTION AND SHA-256 AUTHENTICATION

A Minor Project report submitted
in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

By

DEERAJ MONALA	2203A51496
PRIYA POOSAALA	2203A51512
SRIYA POOSAALA	2203A51513
MAHESH SOORA	2203A51159
PRANAY ALETTI	2203A51533

Under the guidance of

Dr. Balajee maram

Professor, School of CS&AI.



SR University, Ananthasagar, Warangal, Telangana-506371



CERTIFICATE

This is to certify that this project entitled “**SECURE MESSAGING SYSTEM USING AES ENCRYPTION AND SHA-256 AUTHENTICATION**” is the bonafied work carried out by **MAHESH SOORA, PRIYA POOSAALA, SRIYA POOSAALA, DEERAJ MONALA and PRANAY ALETTI** as a Minor Project for the partial fulfillment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year 2024-2025 under our guidance and Supervision.

Dr. Balajee maram

Professor & Dean

SR University

Anathasagar, Warangal

Dr. M. Sheshikala

Professor & Head,

School of CS&AI,

SR University

Anathasagar, Warangal.

TABLE OF CONTENT

S. No.	Title	Page No.
1	INTRODUCTION	5
2	PROBLEM IDENTIFICATION	6
3	REQUIREMENT ANALYSIS	7-8
4	PROPOSED SOLUTION	9-10
5	MODEL TRAINING	11-12
6	ARCHITECTURE DIAGRAM	13
7	FLOW CHART	14
8	DATA FLOW	15-17
9	IMPLEMENTATION	18-20
10	PROGRAM	21-36
11	RESULTS	37-39
12	LEARNING OUTCOME	40-41
13	PROJECT IMPACT	42
14	CONCLUSION	43
15	REFERENCES	44

ABSTRACT

The Secure Messaging System is a robust communication platform designed to ensure privacy and security in real-time digital interactions. Utilizing Advanced Encryption Standard (AES) in CBC mode with PKCS7 padding and SHA-256 hashed authentication, the system guarantees message confidentiality and restricts access to authorized users. Developed in Python, it employs TCP sockets on for reliable client-server communication, complemented by a user-friendly Tkinter-based GUI featuring color-coded message logs and intuitive controls. Chat history persistence enables seamless conversation continuity, while thread-safe operations and comprehensive error handling ensure operational stability. The modular architecture facilitates maintenance and scalability. This project exemplifies the integration of cryptographic techniques, secure authentication, and responsive design, delivering a practical solution for privacy-focused communication in educational and small-scale professional environments.

CHAPTER 1

INTRODUCTION

The rapid growth of digital communication has amplified the need for secure and reliable messaging systems, as cyber threats like eavesdropping and data breaches pose significant risks. This project develops a secure chat application that employs Advanced Encryption Standard (AES) encryption in CBC mode with PKCS7 padding to ensure the confidentiality and integrity of messages. Built in Python, it features a user-friendly Tkinter graphical interface and a client-server architecture using TCP sockets on localhost:12345. A robust authentication feature requires users to log in with credentials (e.g., Risers/12345678), verified against SHA-256 hashed passwords stored in users.json, ensuring only authorized access.

The primary objective is to create a secure, real-time chat system for encrypted communication between a single client and server, supporting username exchange, chat history persistence in chat_history.json, and comprehensive error handling. Messages are encrypted with a 128-bit AES key and a random initialization vector (IV) per message, guaranteeing high security. The GUI provides intuitive controls for managing the server, connecting the client, and exchanging messages, with color-coded logs (green for senders, blue for system, red for errors) and timestamps. Chat history persists across sessions, enabling seamless conversation continuity.

The project adeptly addresses challenges such as thread-safe GUI updates, robust error handling, and graceful disconnection management. By integrating strong authentication, advanced cryptography, and a responsive interface, the system achieves an optimal balance of security and usability. The authentication mechanism ensures that only verified users can access the system, enhancing its reliability. The application's efficient design supports real-time communication with minimal latency, making it ideal for secure interactions. Its modular architecture facilitates easy maintenance and scalability. This project exemplifies the effective application of encryption, networking, authentication, and GUI development, delivering a highly secure and user-friendly communication platform.

CHAPTER 2

PROBLEM IDENTIFICATION

In modern communication systems, ensuring the security and reliability of transmitted data remains a critical challenge. Traditional chat applications often employ unencrypted or weakly encrypted protocols, rendering them susceptible to interception, man-in-the-middle attacks, and unauthorized access. These vulnerabilities can expose sensitive information, resulting in privacy breaches and diminished user trust. Furthermore, many chat systems lack intuitive user interfaces, robust error handling, or mechanisms for persisting conversation history, which hampers their practical usability. The absence of strong access control mechanisms also heightens the risk of unauthorized system entry, compromising overall security.

The development of a secure chat application addresses several specific challenges:

1. **Message Security:** Without robust encryption, messages are vulnerable to interception. Implementing Advanced Encryption Standard (AES) in CBC mode with PKCS7 padding requires meticulous management of 128-bit keys and random initialization vectors (IVs) to ensure uncompromised security.
2. **User Authentication:** Lack of authentication allows unauthorized access to the chat system. Implementing a secure login system using SHA-256 hashed passwords stored in `users.json` ensures only verified users can participate.
3. **Concurrency:** Real-time chat demands concurrent handling of network operations (e.g., message exchange via TCP sockets on `localhost:12345`) and GUI updates, necessitating thread-safe mechanisms to prevent race conditions or interface freezes.
4. **Usability:** A secure system must offer an intuitive Tkinter-based GUI with clear controls for server/client management and color-coded message logs (green for senders, blue for system, red for errors) to ensure accessibility without sacrificing security.
5. **Error Handling:** Network disruptions, decryption failures, or client disconnections require graceful handling to maintain system stability and provide informative feedback to users.
6. **Data Persistence:** Securely storing chat history in `chat_history.json` enables users to access past conversations while maintaining performance and data integrity.

This project addresses these challenges by delivering a secure, user-friendly chat application with AES encryption, SHA-256-based authentication, thread-safe operations, persistent chat history, and comprehensive error handling, creating a reliable and accessible communication platform.

CHAPTER 3

REQUIREMENT ANALYSIS

To develop a secure chat application, a detailed analysis of functional, hardware, software, and feasibility requirements is crucial to ensure robust performance and security.

3.1 Functional Requirements

- **Encryption:** Implement AES-CBC encryption with 128-bit keys and PKCS7 padding to secure messages. Generate random initialization vectors (IVs) for each message to ensure unique ciphertexts.
- **Authentication:** Require user login with credentials (e.g., Risers/12345678), verified against SHA-256 hashed passwords stored in users.json, to prevent unauthorized access.
- **Networking:** Utilize TCP sockets for reliable, real-time communication between a server and a single client on localhost:12345. Exchange usernames upon successful connection.
- **GUI:** Provide a Tkinter-based interface with:
 - Status indicators for server/client connection and authentication states.
 - A scrollable chat area displaying messages, system events, and errors with color-coded logs (green for senders, blue for system, red for errors).
 - Input fields and buttons for sending messages, controlling connectivity, and managing server/client operations.
- **Chat History:** Persist chat history in chat_history.json with timestamps, sender, message content, and message type (normal, system, error) for session continuity.
- **Error Handling:** Manage network errors, decryption failures, authentication failures, and disconnections gracefully, logging errors with informative messages.
- **Concurrency:** Employ threading to handle simultaneous network operations and GUI updates, ensuring a responsive interface without freezes.

3.2 Hardware Requirements

- A standard computer with a multi-core CPU (e.g., Intel i5 or equivalent) to support Python execution and threading.
- Minimum 4GB RAM to accommodate the application and development environment.
- Stable network interface for localhost communication.

3.3 Software Requirements

- **Programming Language:** Python 3.8+ for development.
- **Libraries:**
 - cryptography for AES encryption/decryption.
 - tkinter for GUI development.
 - socket for TCP networking.
 - threading for concurrent operations.
 - json for chat history and user credential persistence.
 - hashlib for SHA-256 password hashing.
 - datetime for timestamp generation.
- **Development Environment:** Any Python IDE (e.g., PyCharm, VS Code) or Jupyter Notebook for coding and testing.
- **Operating System:** Windows, macOS, or Linux, ensuring cross-platform compatibility.

3.4 Feasibility Analysis

- **Technical Feasibility:** Python's comprehensive libraries (cryptography, tkinter, socket, hashlib) streamline implementation. Authentication, threading, and JSON handling are well-supported, ensuring technical viability.
- **Operational Feasibility:** The intuitive Tkinter GUI, with clear controls (e.g., Login, Start/Stop Server, Connect/Disconnect) and color-coded logs, enhances user accessibility. The system's modular design supports maintenance and scalability.
- **Economic Feasibility:** Utilizing open-source tools and libraries eliminates software costs. Minimal hardware requirements make the project cost-effective for development, testing, and deployment.

CHAPTER 4

PROPOSED SOLUTION

To address the identified challenges, we propose a secure chat application that integrates AES encryption, user authentication, a client-server architecture, and a Tkinter-based GUI to deliver a robust and user-friendly communication platform. The system ensures message confidentiality, secure access control, real-time communication, and an intuitive user experience.

Key Features of the Proposed System

1. Secure Communication:

- Messages are encrypted using AES-CBC with a 128-bit key and random 16-byte initialization vectors (IVs), ensuring confidentiality.
- PKCS7 padding aligns messages with the AES block size.
- Decryption errors are logged in red to alert users of issues (e.g., corrupted data).

2. User Authentication:

- Users must log in with credentials (e.g., Risers/12345678), verified against SHA-256 hashed passwords stored in users.json, preventing unauthorized access.
- A Tkinter-based login dialog ensures secure and user-friendly credential entry.

3. Client-Server Architecture:

- The server listens on localhost:12345 and supports a single client connection at a time.
- Upon connection, the server and client exchange usernames for personalized message display.
- Threading enables concurrent message sending/receiving and connection management.

4. User-Friendly GUI:

- The server GUI includes buttons to start/stop the server, a status label (Stopped, Waiting, Connected), and a scrollable chat area.
- The client GUI offers connect/disconnect buttons, a new client launch option, and a status label (Disconnected, Connected).

- Both interfaces display messages with color-coded tags: sender names (green), system messages (blue), errors (red), and timestamps for clarity.

5. Chat History Persistence:

- Messages are saved in chat_history.json with timestamps, sender, message content, and type (normal, system, error), loaded on client startup.
- History updates in real-time and is saved on disconnection or window close.

6. Error Handling and Robustness:

- Graceful handling of socket errors, connection resets, authentication failures, and decryption issues with informative logging.
- Thread-safe GUI updates using master.after prevent crashes and ensure responsiveness.
- Automatic detection of client disconnections with corresponding UI updates.

This solution effectively combines authentication, encryption, usability, and robustness, delivering a secure and reliable chat platform optimized for single-client communication.

CHAPTER 5

MODEL TRAINING

In this project, "model training" refers to the development, testing, and refinement of the secure chat application's core components: encryption, authentication, networking, and GUI functionality. This iterative process ensures optimal performance and reliability.

5.1 Development Process

1. Cryptographic Module:

- Built `encrypt_message` and `decrypt_message` using cryptography for AES-CBC with 128-bit keys and random IVs.
- Tested padding, IV handling, and error logging for invalid ciphertexts.

2. Authentication Module:

- Implemented login with credentials verified via SHA-256 hashes in `users.json`.
- Tested valid/invalid logins and Tkinter dialog usability.

3. Networking Module:

- Set up TCP sockets on `localhost:12345` for server-client communication.
- Tested username exchange and messaging under normal and error conditions.
- Used threading for concurrent operations without GUI blocking.

4. GUI Module:

- Designed Tkinter interfaces with status indicators, chat areas, and controls.
- Tested thread-safe updates with `master.after` for responsiveness.

5.2 Testing and Validation

• Unit Testing:

- Tested encryption, authentication, sockets, and `chat_history.json` with edge cases.

• Integration Testing:

- Validated end-to-end flow: authentication, encryption, transmission, and display.
- Tested disconnections and authentication failures for cleanup and UI updates.

• Usability Testing:

- Ensured GUI responsiveness and clear logging (green: senders, blue: system, red: errors).

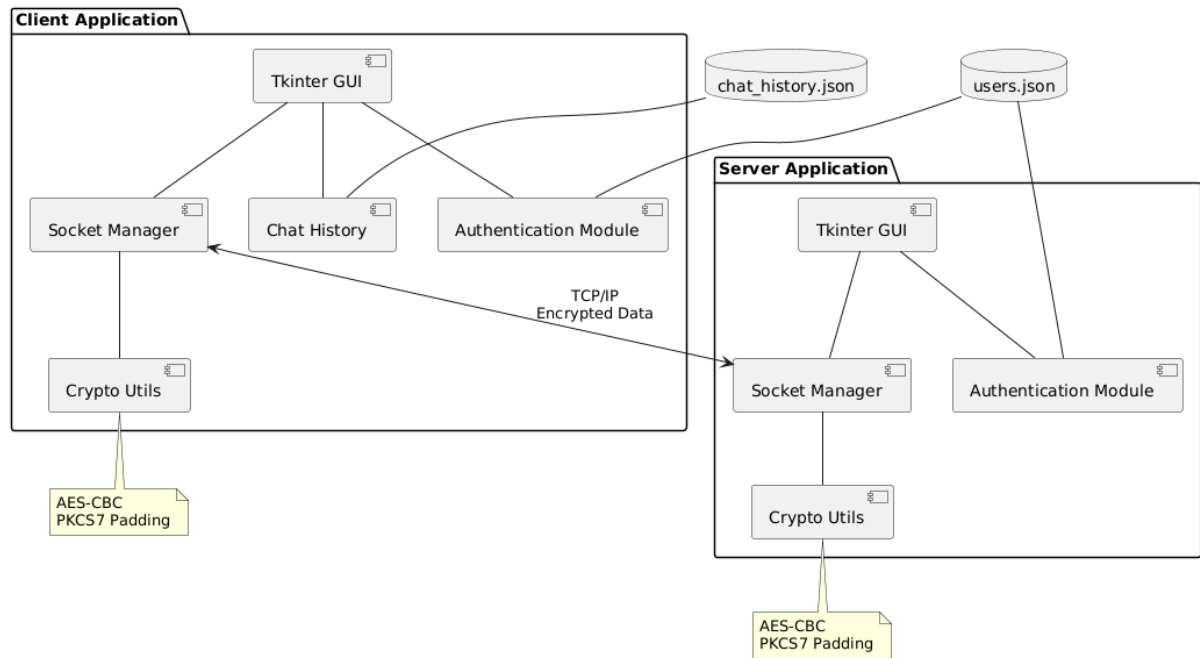
5.3 Optimization

- Optimized socket timeouts and GUI updates for efficiency.
- Streamlined JSON handling to prevent corruption during rapid saves.

This process delivers a secure, reliable, and user-friendly chat application with robust authentication and encryption.

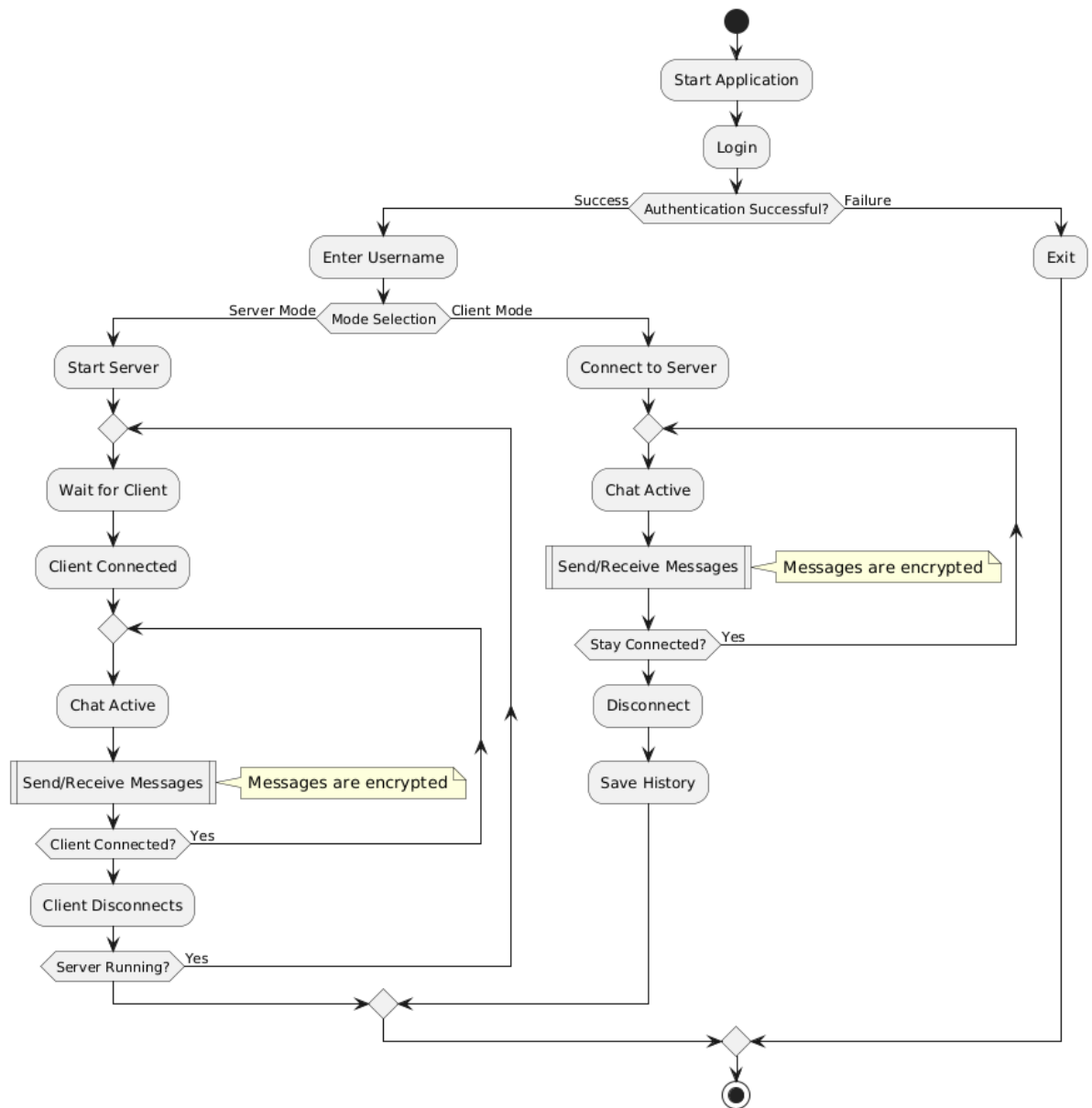
CHAPTER 6

ARCHITECTURE DIAGRAM



CHAPTER 7

FLOW CHART



CHAPTER 8

DATA FLOW

The data flow in the secure chat application outlines the processing, encryption, transmission, and display of messages and system events, with persistence in a JSON file and secure user authentication. The following steps describe the current data flow:

8.1 Initialization

- **Server:**
 - The server initializes a Tkinter login dialog, requiring credentials (e.g., Risers/12345678), verified against SHA-256 hashed passwords in users.json.
 - Upon successful authentication, the GUI prompts for a display username (default: "Server").
 - The Start Server button creates a socket, binds to localhost:12345, and listens for a client.
- **Client:**
 - The client initializes a Tkinter login dialog for authentication with the same credentials.
 - After authentication, the GUI prompts for a display username (default: "Client").
 - The Connect button or --auto-connect flag initiates a connection to the server.
 - The client loads and displays chat history from chat_history.json in the chat area.

8.2 Connection Establishment

- The server accepts a client connection and receives the client's username.
- The server sends its username to the client.
- Both update GUI status (server: "Connected to [client_username]"; client: "Connected to [server_username]").
- A system message is logged (e.g., "Client connected: [username]").

8.3 Message Sending

- The user types a message in the GUI input field and presses Send or Enter.
- The encrypt_message function:
 - Encodes the message to UTF-8.

- Applies PKCS7 padding for AES block size alignment.
- Generates a random 16-byte IV.
- Encrypts the padded message using AES-CBC with a hardcoded key.
- Concatenates IV and ciphertext.
- The encrypted message is sent via the socket to the recipient.
- The sender logs the plaintext message in the GUI with a timestamp and sender name.

8.4 Message Receiving

- The recipient's socket receives the encrypted message (IV + ciphertext).
- The `decrypt_message` function:
 - Extracts the 16-byte IV and ciphertext.
 - Decrypts the ciphertext using AES-CBC with the shared key.
 - Removes PKCS7 padding.
 - Decodes the plaintext to UTF-8.
- If decryption fails, an error (e.g., "[Decryption Error: Invalid padding]") is logged in red.
- The decrypted message is logged in the GUI with a timestamp and sender name.

8.5 Chat History Persistence

- Messages, system events, and errors are added to the client's in-memory `chat_history` list as JSON objects with fields: timestamp, message, sender, and type (normal, system, error).
- History is saved to `chat_history.json` on client disconnection or window close.
- On startup, the client loads and displays history with a header ("--- Chat History ---") and footer.

8.6 Error Handling and Disconnection

- Network errors (e.g., `ConnectionResetError`) trigger disconnection, logging a system message (e.g., "Connection reset by client").
- Authentication failures exit the application with an error message.
- The server reverts to "Waiting for connection" and closes the client socket.
- The client updates to "Disconnected" and disables input fields.
- Decryption errors are logged without disrupting the connection unless critical.

8.7 Threading and Concurrency

- The server uses an `accept_thread` to listen for client connections.
- A `receive_thread` handles incoming messages to avoid GUI blocking.
- The client uses a `receive_thread` for incoming messages.
- Thread-safe GUI updates use `master.after` to schedule chat area updates.

This data flow ensures secure, authenticated, and reliable communication, with messages encrypted, logged, and persisted, delivering a seamless user experience.

CHAPTER 9

IMPLEMENTATION

The implementation of the secure chat application integrates cryptographic, authentication, networking, and graphical user interface (GUI) components to create a robust, secure, and user-friendly system. This section details the development process ensuring seamless encrypted communication between a server and a client.

9.1 Environment Setup

The development environment uses Python 3.8+, leveraging its robust library ecosystem. Key libraries include:

- `cryptography`: Implements AES-CBC encryption/decryption with PKCS7 padding.
- `tkinter`: Builds the GUI framework.
- `socket`: Facilitates TCP-based client-server communication.
- `threading`: Enables concurrent network and GUI operations.
- `json`: Manages chat history and user credential persistence.
- `hashlib`: Supports SHA-256 password hashing.
- `datetime`: Generates message timestamps.

The environment runs on Windows, macOS, or Linux, using IDEs like PyCharm or VS Code. Dependencies are installed via pip, requiring minimal hardware (4GB RAM, multi-core CPU).

9.2 Code Structure and Modularity

The application is organized into three Python modules for maintainability:

- `crypto_utils.py`: Contains `encrypt_message` and `decrypt_message` for AES encryption/decryption.
- `chat_server.py`: Implements the `ChatServer` class, handling server-side GUI, networking, and authentication.
- `chat_client.py`: Implements the `ChatClient` class, managing client-side GUI, networking, and authentication.

This modular structure separates concerns, enhancing clarity and scalability.

9.3 Cryptographic Implementation

The cryptographic module secures message transmission:

- **Encryption:** `encrypt_message` encodes messages to UTF-8, applies PKCS7 padding, generates a random 16-byte IV using `os.urandom`, and encrypts with AES-CBC using a 128-bit key. The IV and ciphertext are concatenated.
- **Decryption:** `decrypt_message` extracts the IV, decrypts the ciphertext, removes padding, and decodes the plaintext, logging errors for invalid inputs.
- **Key Note:** A hardcoded key is used, with plans for future secure key exchange.

Testing validated encryption/decryption with edge cases (empty messages, special characters).

9.4 Authentication Implementation

Authentication ensures secure access:

- A Tkinter login dialog prompts for credentials (e.g., Risers/12345678), verified against SHA-256 hashed passwords in `users.json`.
- The `authenticate_user` method hashes the input password and compares it with the stored hash, exiting on failure.
- Testing confirmed robust handling of valid/invalid credentials and dialog usability.

9.5 Networking Implementation

Networking uses TCP sockets:

- **Server:** `ChatServer` binds to `localhost:12345`, listens for one client, and uses threads (`accept_thread`, `receive_thread`) for connections and messages. It exchanges usernames post-connection.
- **Client:** `ChatClient` connects to the server, exchanges usernames, and uses a `receive_thread` for messages.
- **Error Handling:** Manages network errors (e.g., `ConnectionResetError`) by logging, closing sockets, and updating the GUI.

Tests verified reliable communication and recovery from disconnections.

9.6 GUI Implementation

The Tkinter GUI enhances usability:

- **Server GUI:**
 - Displays status (e.g., "Waiting", "Connected to [client_username]").
 - Includes Start/Stop Server button, scrollable chat area, and input field with Send button.
 - Prompts for authentication and display username (default: "Server").

- **Client GUI:**
 - Shows status (e.g., "Disconnected", "Connected to [server_username]").
 - Features Connect/Disconnect, New Client buttons, chat area, and input field.
 - Loads chat_history.json on startup.
- **Message Display:** Color-coded logs (green: senders, blue: system, red: errors) with timestamps.
- **Thread Safety:** Uses master.after for responsive updates.

Usability tests confirmed intuitive controls and clear feedback.

9.7 Chat History Persistence

The client persists chat history in chat_history.json:

- Entries include timestamp, message, sender, and type (normal, system, error).
- History is loaded on startup, displayed with header/footer, and saved on disconnection/window close.
- Optimized file operations handle rapid updates without corruption.

Testing ensured integrity with large logs.

9.8 Testing and Validation

Rigorous testing ensured reliability:

- **Unit Testing:** Validated encryption, authentication, sockets, and JSON handling with edge cases.
- **Integration Testing:** Verified end-to-end flow (authentication, encryption, transmission, display) with simultaneous server-client operation.
- **Usability Testing:** Confirmed GUI responsiveness, clear error logging, and ease of use.

The implementation delivers a secure, authenticated, and user-friendly chat system, meeting all project objectives.

CHAPTER 10

PROGRAM

1) Encryption/Decryption :

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding
import os
import base64
KEY = b'Sixteen byte key'
def encrypt_message(message):
    iv = os.urandom(16)
    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(message.encode('utf-8')) + padder.finalize()
    cipher = Cipher(algorithms.AES(KEY), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    encrypted = encryptor.update(padded_data) + encryptor.finalize()
    return iv + encrypted
def decrypt_message(ciphertext):
    try:
        iv = ciphertext[:16]
        actual_cipher = ciphertext[16:]
        cipher = Cipher(algorithms.AES(KEY), modes.CBC(iv), backend=default_backend())
        decryptor = cipher.decryptor()
        decrypted_padded = decryptor.update(actual_cipher) + decryptor.finalize()
        unpadder = padding.PKCS7(128).unpadder()
        decrypted = unpadder.update(decrypted_padded) + unpadder.finalize()
        return decrypted.decode('utf-8')
    except Exception as e:
        return f"[Decryption Error: {str(e)}]"
```

2) Server-Side code:

```
import socket

import threading

import tkinter as tk

from tkinter import scrolledtext, simpledialog, messagebox

from datetime import datetime

from crypto_utils import encrypt_message, decrypt_message

import json

import hashlib

class ChatServer:

    def __init__(self, master):

        self.master = master

        self.master.title("🔒 Secure Chat - Server")

        self.master.geometry("600x500")

        self.master.protocol("WM_DELETE_WINDOW", self.on_closing)

        if not self.authenticate_user():

            messagebox.showerror("Authentication Failed", "Invalid username or password. Exiting.")

            self.master.destroy()

            return

        self.username = simpledialog.askstring("Username", "Enter your display username:",

parent=master)

        if not self.username:

            self.username = "Server"

        self.status_frame = tk.Frame(master)

        self.status_frame.pack(fill=tk.X, padx=10, pady=5)

        self.status_label = tk.Label(self.status_frame, text="Status: Server Stopped", fg="red")

        self.status_label.pack(side=tk.LEFT)

        self.server_button = tk.Button(self.status_frame, text="Start Server",

            command=self.toggle_server)

        self.server_button.pack(side=tk.RIGHT)
```

```

self.text_area = scrolledtext.ScrolledText(master, wrap=tk.WORD, width=60, height=20, state='disabled')

self.text_area.pack(padx=10, pady=10, fill=tk.BOTH, expand=True)

self.input_frame = tk.Frame(master)

self.input_frame.pack(fill=tk.X, padx=10, pady=5)

self.entry = tk.Entry(self.input_frame, width=50)

self.entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 5))

self.entry.bind("<Return>", self.send_message)

self.entry.config(state=tk.DISABLED)

self.send_button = tk.Button(self.input_frame, text="Send", command=self.send_message, state=tk.DISABLED)

self.send_button.pack(side=tk.RIGHT)

self.sock = None

self.conn = None

self.addr = None

self.client_username = None

self.server_running = False

self.client_connected = False

self.accept_thread = None

def authenticate_user(self):
    try:
        with open("users.json", "r") as f:
            users_data = json.load(f)

            users = {user["username"]: user["password_hash"] for user in users_data["users"]}
    except FileNotFoundError:
        print("Error: users.json not found.")
        return False

    auth_dialog = tk.Toplevel(self.master)
    auth_dialog.title("Login")
    auth_dialog.geometry("300x150")
    auth_dialog.transient(self.master)

```

```

auth_dialog.grab_set()

tk.Label(auth_dialog, text="Username:").pack(pady=5)

username_entry = tk.Entry(auth_dialog)

username_entry.pack(pady=5)

tk.Label(auth_dialog, text="Password:").pack(pady=5)

password_entry = tk.Entry(auth_dialog, show="*")

password_entry.pack(pady=5)

result = {"authenticated": False}

def verify():

    username = username_entry.get().strip()

    password = password_entry.get().strip()

    if username in users:

        input_hash = hashlib.sha256(password.encode()).hexdigest()

        if input_hash == users[username]:

            result["authenticated"] = True

    auth_dialog.destroy()

tk.Button(auth_dialog, text="Login", command=verify).pack(pady=10)

self.master.wait_window(auth_dialog)

return result["authenticated"]

def toggle_server(self):

    if not self.server_running:

        self.start_server()

    else:

        self.stop_server()

def start_server(self):

    try:

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        self.sock.bind(('localhost', 12345))

        self.sock.listen(1)

```

```

        self.server_running = True

        self.status_label.config(text="Status: Waiting for connection", fg="orange")

        self.server_button.config(text="Stop Server")

        self.log_message("Server started. Waiting for connection...", message_type="system")

        self.accept_thread = threading.Thread(target=self.accept_connections, daemon=True)

        self.accept_thread.start()

    except Exception as e:

        messagebox.showerror("Server Error", f"Failed to start server: {str(e)}")

        self.stop_server()

def stop_server(self):

    if self.client_connected:

        self.disconnect_client()

    if self.sock:

        try:

            self.sock.close()

        except:

            pass

    self.server_running = False

    self.status_label.config(text="Status: Server Stopped", fg="red")

    self.server_button.config(text="Start Server")

    self.entry.config(state=tk.DISABLED)

    self.send_button.config(state=tk.DISABLED)

    self.log_message("Server stopped", message_type="system")

def accept_connections(self):

    while self.server_running:

        try:

            self.sock.settimeout(1.0)

            try:

                self.conn, self.addr = self.sock.accept()

            except socket.timeout:


```

```

        continue

    self.sock.settimeout(None)

    self.client_username = self.conn.recv(1024).decode()

    self.conn.send(self.username.encode())

    self.client_connected = True

    self.status_label.config(text=f"Connected to: {self.client_username}", fg="green")

    self.entry.config(state=tk.NORMAL)

    self.send_button.config(state=tk.NORMAL)

    self.log_message(f"Client connected: {self.client_username} from {self.addr}",
                    message_type="system")

    receive_thread = threading.Thread(target=self.receive_messages, daemon=True)

    receive_thread.start()

    receive_thread.join()

except Exception as e:

    if self.server_running:

        self.log_message(f"Error in connection: {str(e)}", message_type="error")

    finally:

        self.disconnect_client()

def disconnect_client(self):

    if self.conn:

        try:

            self.conn.close()

        except:

            pass

    self.client_connected = False

    self.conn = None

    self.client_username = None

    if self.server_running:

        self.status_label.config(text="Status: Waiting for connection", fg="orange")

    self.entry.config(state=tk.DISABLED)

```

```

self.send_button.config(state=tk.DISABLED)

if self.server_running:

    self.log_message("Client disconnected. Waiting for new connection.",
        message_type="system")

def send_message(self, event=None):

    message = self.entry.get().strip()

    if message and self.client_connected:

        try:

            encrypted = encrypt_message(message)

            self.conn.send(encrypted)

            self.log_message(message, sender=self.username)

            self.entry.delete(0, tk.END)

        except Exception as e:

            self.log_message(f"Failed to send message: {str(e)}", message_type="error")

            self.disconnect_client()

def receive_messages(self):

    while self.client_connected:

        try:

            data = self.conn.recv(1024)

            if not data:

                break

            decrypted = decrypt_message(data)

            self.log_message(decrypted, sender=self.client_username)

        except ConnectionResetError:

            self.log_message("Connection was reset by the client", message_type="error")

            break

        except Exception as e:

            self.log_message(f"Error receiving message: {str(e)}", message_type="error")

            break

def log_message(self, message, sender=None, message_type="normal"):

```

```

timestamp = datetime.now().strftime('%H:%M:%S')

self.master.after(0, self._update_text_area, message, sender, timestamp, message_type)

def _update_text_area(self, message, sender, timestamp, message_type):
    self.text_area.config(state='normal')

    if message_type == "error":
        self.text_area.insert(tk.END, f"[ERROR] ({timestamp}): {message}\n", "error")
        self.text_area.tag_configure("error", foreground="red")

    elif message_type == "system":
        self.text_area.insert(tk.END, f"[SYSTEM] ({timestamp}): {message}\n", "system")
        self.text_area.tag_configure("system", foreground="blue")

    else:
        if sender:
            self.text_area.insert(tk.END, f"{sender} ({timestamp}): ", "sender")
            self.text_area.tag_configure("sender", foreground="green")
            self.text_area.insert(tk.END, f"{message}\n")
        else:
            self.text_area.insert(tk.END, f"({timestamp}): {message}\n")

    self.text_area.see(tk.END)
    self.text_area.config(state='disabled')

def on_closing(self):
    self.stop_server()
    self.master.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = ChatServer(root)
    root.mainloop()

```

3) Client-Side code:

```
import socket
import threading
import tkinter as tk
from tkinter import scrolledtext, simpledialog, messagebox
from datetime import datetime
from crypto_utils import encrypt_message, decrypt_message
import json
import os
import sys
import hashlib

class ChatClient:
    def __init__(self, master, auto_connect=False):
        self.master = master

        self.master.title("🔒 Secure Chat - Client")
        self.master.geometry("600x500")
        self.master.protocol("WM_DELETE_WINDOW", self.on_closing)

        if not self.authenticate_user():
            messagebox.showerror("Authentication Failed", "Invalid username or password. Exiting.")
            self.master.destroy()
            return

        self.chat_history = []
        self.history_file = "chat_history.json"
        self.load_chat_history()

        self.username = simpledialog.askstring("Username", "Enter your display username:",
parent=master)

        if not self.username:
            self.username = "Client"

        self.status_frame = tk.Frame(master)
        self.status_frame.pack(fill=tk.X, padx=10, pady=5)
```

```

self.status_label = tk.Label(self.status_frame, text="Status: Disconnected", fg="red")

self.status_label.pack(side=tk.LEFT)

self.connection_button = tk.Button(self.status_frame, text="Connect",
command=self.connect_to_server)

self.connection_button.pack(side=tk.RIGHT)

self.new_client_button = tk.Button(self.status_frame, text="New Client",
command=self.new_client, state=tk.DISABLED)

self.new_client_button.pack(side=tk.RIGHT, padx=5)


self.text_area = scrolledtext.ScrolledText(master, wrap=tk.WORD, width=60, height=20,
state='disabled')

self.text_area.pack(padx=10, pady=10, fill=tk.BOTH, expand=True)

self.input_frame = tk.Frame(master)

self.input_frame.pack(fill=tk.X, padx=10, pady=5)

self.entry = tk.Entry(self.input_frame, width=50)

self.entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 5))

self.entry.bind("<Return>", self.send_message)

self.entry.config(state=tk.DISABLED)

self.send_button = tk.Button(self.input_frame, text="Send", command=self.send_message,
state=tk.DISABLED)

self.send_button.pack(side=tk.RIGHT)

self.sock = None

self.server_username = None

self.connected = False

self.display_chat_history()

if auto_connect:

    self.master.after(100, self.connect_to_server)

def authenticate_user(self):

    try:

        with open("users.json", "r") as f:

            users_data = json.load(f)

```

```

        users = {user["username"]: user["password_hash"] for user in users_data["users"]}
except FileNotFoundError:

    print("Error: users.json not found.")

    return False

auth_dialog = tk.Toplevel(self.master)
auth_dialog.title("Login")
auth_dialog.geometry("300x150")
auth_dialog.transient(self.master)
auth_dialog.grab_set()

tk.Label(auth_dialog, text="Username:").pack(pady=5)

username_entry = tk.Entry(auth_dialog)
username_entry.pack(pady=5)

tk.Label(auth_dialog, text="Password:").pack(pady=5)

password_entry = tk.Entry(auth_dialog, show="*")
password_entry.pack(pady=5)

result = {"authenticated": False}

def verify():

    username = username_entry.get().strip()

    password = password_entry.get().strip()

    if username in users:

        input_hash = hashlib.sha256(password.encode()).hexdigest()

        if input_hash == users[username]:

            result["authenticated"] = True

        auth_dialog.destroy()

tk.Button(auth_dialog, text="Login", command=verify).pack(pady=10)

self.master.wait_window(auth_dialog)

return result["authenticated"]

def load_chat_history(self):

    try:

        if os.path.exists(self.history_file):

```

```

        with open(self.history_file, 'r') as f:
            self.chat_history = json.load(f)
except Exception as e:
    print(f"Error loading chat history: {e}")
    self.chat_history = []
def save_chat_history(self):
    try:
        with open(self.history_file, 'w') as f:
            json.dump(self.chat_history, f)
    except Exception as e:
        print(f"Error saving chat history: {e}")
def display_chat_history(self):
    if not self.chat_history:
        return
    self.text_area.config(state='normal')
    self.text_area.insert(tk.END, "--- Chat History ---\n", "history_header")
    self.text_area.tag_configure("history_header", foreground="purple")
    for entry in self.chat_history:
        timestamp = entry.get('timestamp', "")
        message = entry.get('message', "")
        sender = entry.get('sender', "")
        message_type = entry.get('type', 'normal')
        if message_type == "error":
            self.text_area.insert(tk.END, f"[ERROR] ({timestamp}): {message}\n", "error")
            self.text_area.tag_configure("error", foreground="red")
        elif message_type == "system":
            self.text_area.insert(tk.END, f"[SYSTEM] ({timestamp}): {message}\n", "system")
            self.text_area.tag_configure("system", foreground="blue")
        else:
            if sender:

```

```

        self.text_area.insert(tk.END, f'{{sender}} ({{timestamp}}): ', "sender")

        self.text_area.tag_configure("sender", foreground="green")

        self.text_area.insert(tk.END, f'{{message}}\n')

    else:

        self.text_area.insert(tk.END, f'({{timestamp}}): {{message}}\n')

    self.text_area.insert(tk.END, "--- End of History ---\n\n", "history_footer")

    self.text_area.tag_configure("history_footer", foreground="purple")

    self.text_area.see(tk.END)

    self.text_area.config(state='disabled')

def connect_to_server(self):

    if self.connected:

        return

    try:

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.sock.connect(('localhost', 12345))

        self.connected = True

        self.sock.send(self.username.encode())

        self.server_username = self.sock.recv(1024).decode()

        self.status_label.config(text=f'Connected to: {self.server_username}', fg="green")

        self.connection_button.config(text="Disconnect", command=self.disconnect_from_server)

        self.new_client_button.config(state=tk.DISABLED)

        self.entry.config(state=tk.NORMAL)

        self.send_button.config(state=tk.NORMAL)

        self.log_message(f'Connected to {self.server_username}!', message_type="system")

        threading.Thread(target=self.receive_messages, daemon=True).start()

    except ConnectionRefusedError:

        messagebox.showerror("Error", "Could not connect to the server.")

    except Exception as e:

        messagebox.showerror("Error", f'Connection error: {str(e)}')

def disconnect_from_server(self):

```

```

if self.sock:

    try:

        self.sock.close()

    except:

        pass

self.connected = False

self.status_label.config(text="Status: Disconnected", fg="red")

self.connection_button.config(text="Connect", command=self.connect_to_server)

self.new_client_button.config(state=tk.NORMAL)

self.entry.config(state=tk.DISABLED)

self.send_button.config(state=tk.DISABLED)

self.log_message("Disconnected from server", message_type="system")


self.save_chat_history()

def new_client(self):

    self.save_chat_history()

    python = sys.executable

    os.execl(python, python, *sys.argv, "--auto-connect")

def send_message(self, event=None):

    message = self.entry.get().strip()

    if message and self.connected:

        try:

            encrypted = encrypt_message(message)

            self.sock.send(encrypted)

            self.log_message(message, sender=self.username)

            self.entry.delete(0, tk.END)

        except Exception as e:

            self.log_message(f"Failed to send message: {str(e)}", message_type="error")

            self.disconnect_from_server()

def receive_messages(self):

```

```

while self.connected:

    try:

        data = self.sock.recv(1024)

        if not data:

            break

        decrypted = decrypt_message(data)

        self.log_message(decrypted, sender=self.server_username)

    except ConnectionResetError:

        self.log_message("Connection was reset by the server", message_type="error")

        break

    except Exception as e:

        self.log_message(f"Error receiving message: {str(e)}", message_type="error")

        break


if self.connected:

    self.master.after(100, self.disconnect_from_server)

def log_message(self, message, sender=None, message_type="normal"):

    timestamp = datetime.now().strftime("%H:%M:%S")

    history_entry = {

        'timestamp': timestamp,

        'message': message,

        'sender': sender,

        'type': message_type

    }

    self.chat_history.append(history_entry)

    self.text_area.config(state='normal')

    if message_type == "error":

        self.text_area.insert(tk.END, f"[ERROR] ({timestamp}): {message}\n", "error")

        self.text_area.tag_configure("error", foreground="red")

    elif message_type == "system":

```

```

        self.text_area.insert(tk.END, f"[SYSTEM] ( {timestamp}): {message}\n", "system")

        self.text_area.tag_configure("system", foreground="blue")

    else:

        if sender:

            self.text_area.insert(tk.END, f" {sender} ( {timestamp}): ", "sender")

            self.text_area.tag_configure("sender", foreground="green")

            self.text_area.insert(tk.END, f" {message}\n")

        else:

            self.text_area.insert(tk.END, f" ( {timestamp}): {message}\n")

    self.text_area.see(tk.END)

    self.text_area.config(state='disabled')

def on_closing(self):

    self.disconnect_from_server()

    self.save_chat_history()

    self.master.destroy()

if __name__ == "__main__":

    root = tk.Tk()

    auto_connect = "--auto-connect" in sys.argv

    app = ChatClient(root, auto_connect)

    root.mainloop()

```

4) Users.json:

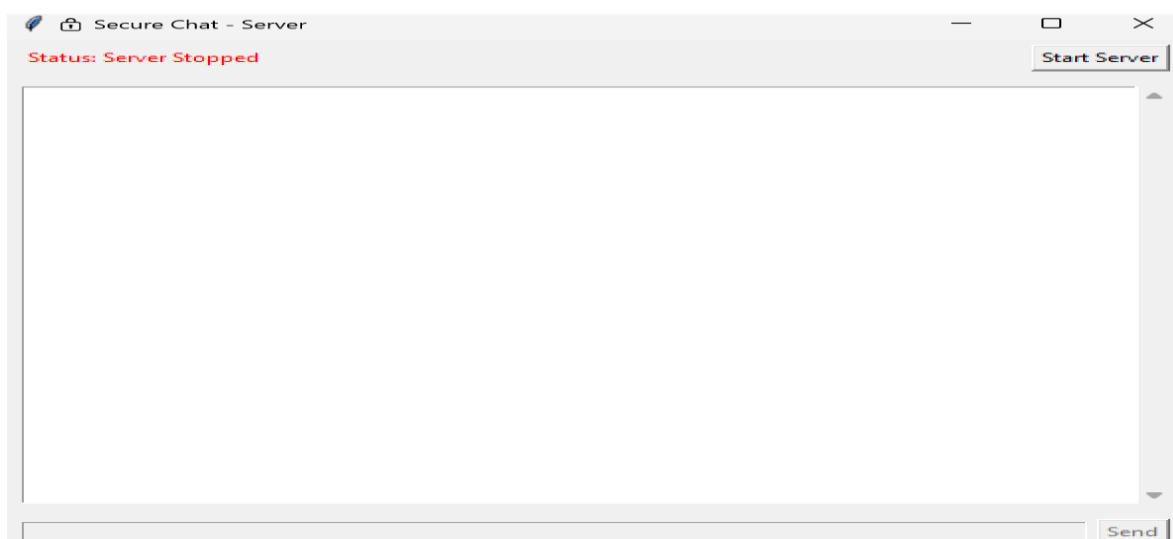
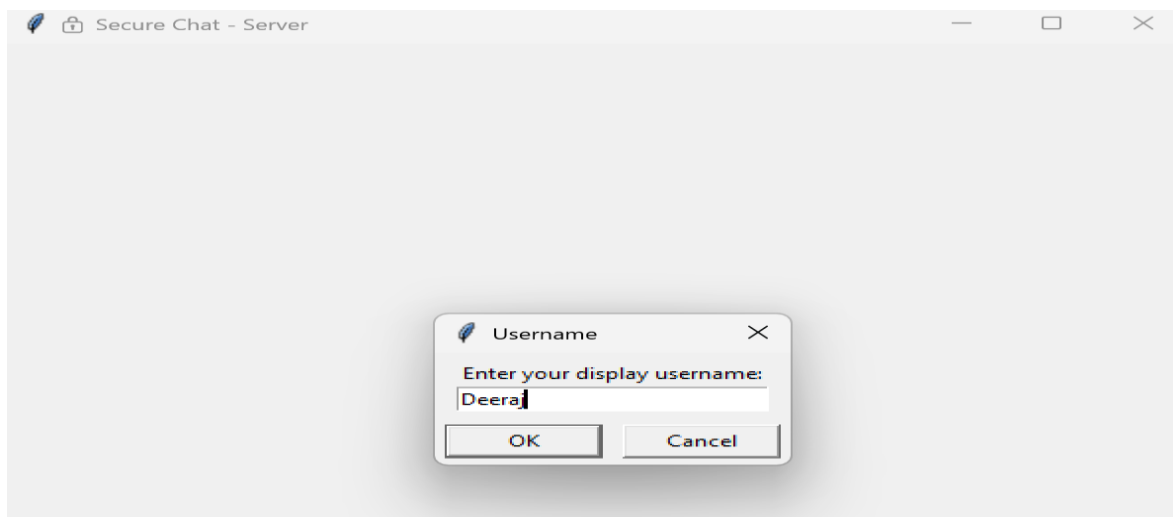
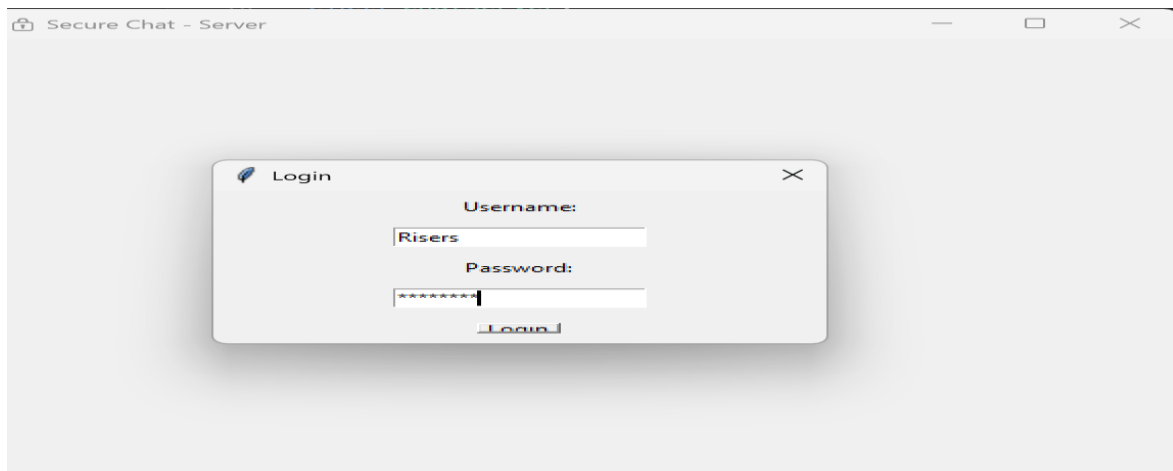
```

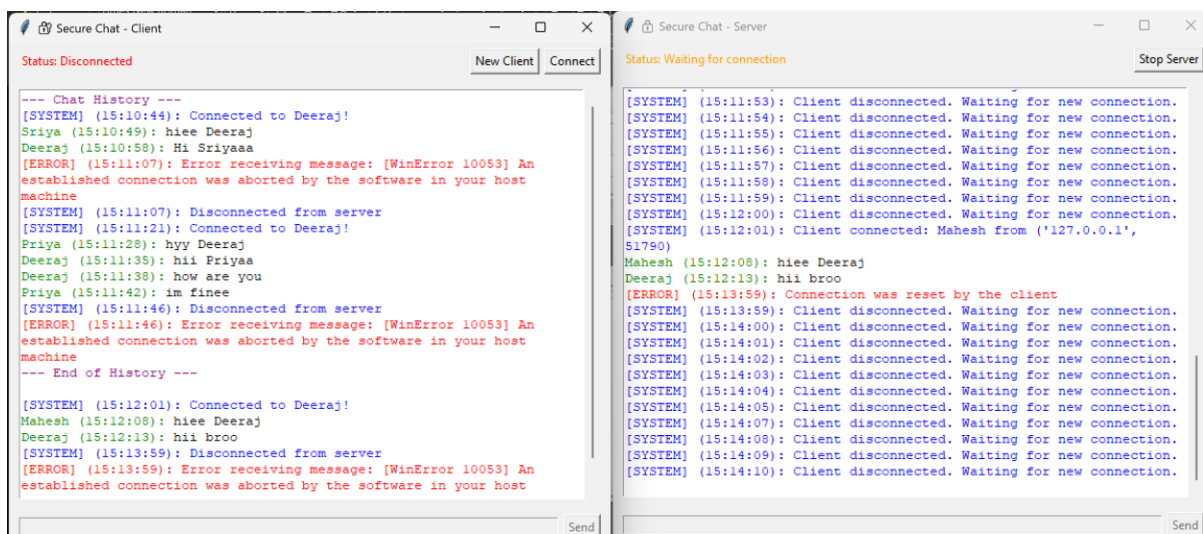
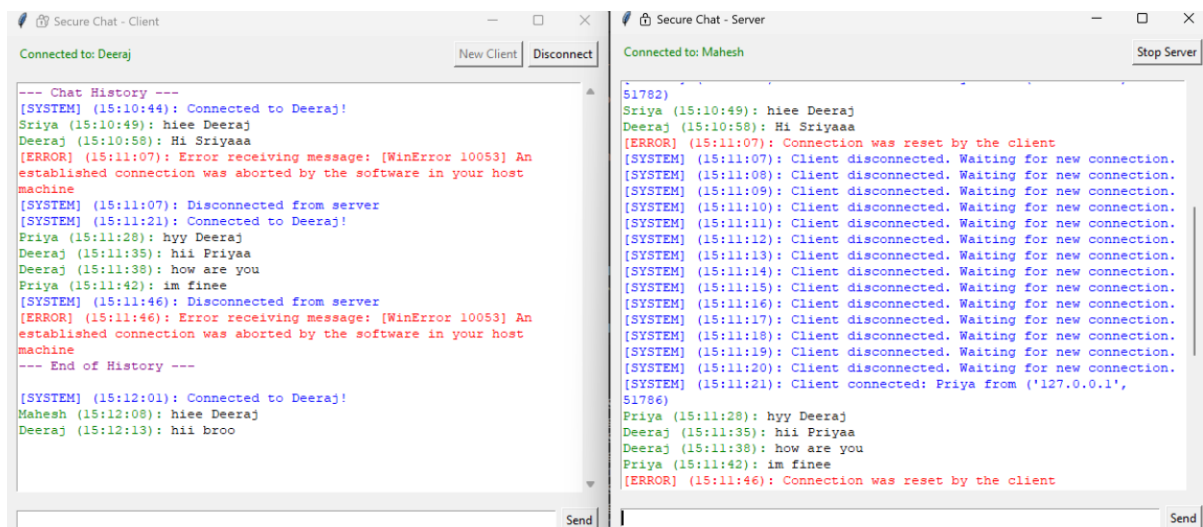
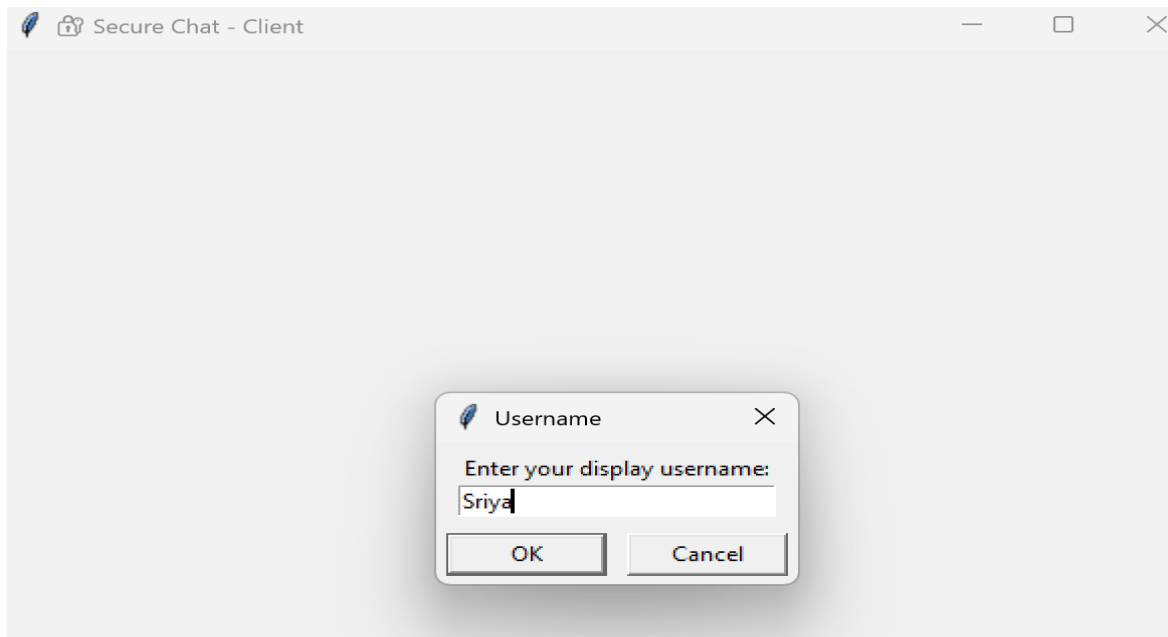
{
  "users": [
    {
      "username": "Risers", "password_hash":
      "ef797c8118f02dfb649607dd5d3f8c7623048c9c063d532cc95c5ed7a898a64f"
    }
  ]
}

```

CHAPTER 11

RESULTS





Chat history:

```
[{"timestamp": "15:10:44", "message": "Connected to Deeraj!", "sender": null, "type": "system"}, {"timestamp": "15:10:49", "message": "hiee Deeraj", "sender": "Sriya", "type": "normal"}, {"timestamp": "15:10:58", "message": "Hi Sriyaaa", "sender": "Deeraj", "type": "normal"}, {"timestamp": "15:11:07", "message": "Error receiving message: [WinError 10053] An established connection was aborted by the software in your host machine", "sender": null, "type": "error"}, {"timestamp": "15:11:07", "message": "Disconnected from server", "sender": null, "type": "system"}, {"timestamp": "15:11:21", "message": "Connected to Deeraj!", "sender": null, "type": "system"}, {"timestamp": "15:11:28", "message": "hyy Deeraj", "sender": "Priya", "type": "normal"}, {"timestamp": "15:11:35", "message": "hii Priyaa", "sender": "Deeraj", "type": "normal"}, {"timestamp": "15:11:38", "message": "how are you", "sender": "Deeraj", "type": "normal"}, {"timestamp": "15:11:42", "message": "im finee", "sender": "Priya", "type": "normal"}, {"timestamp": "15:11:46", "message": "Disconnected from server", "sender": null, "type": "system"}, {"timestamp": "15:11:46", "message": "Error receiving message: [WinError 10053] An established connection was aborted by the software in your host machine", "sender": null, "type": "error"}, {"timestamp": "15:12:01", "message": "Connected to Deeraj!", "sender": null, "type": "system"}, {"timestamp": "15:12:08", "message": "hiee Deeraj", "sender": "Mahesh", "type": "normal"}, {"timestamp": "15:12:13", "message": "hii broo", "sender": "Deeraj", "type": "normal"}, {"timestamp": "15:13:59", "message": "Error receiving message: [WinError 10053] An established connection was aborted by the software in your host machine", "sender": null, "type": "error"}, {"timestamp": "15:13:59", "message": "Disconnected from server", "sender": null, "type": "system"}, {"timestamp": "15:14:24", "message": "Disconnected from server", "sender": null, "type": "system"}]
```

CHAPTER 12

LEARNING OUTCOME

The development of the secure chat application provided significant learning experiences in cryptography, authentication, networking, GUI design, and software engineering. The following outcomes highlight the skills and insights gained:

12.1 Understanding Cryptographic Techniques

Implementing AES-CBC encryption with PKCS7 padding deepened our knowledge of symmetric encryption and the role of random IVs in ensuring unique ciphertexts. We recognized the limitations of hardcoded keys, prompting exploration of secure key management practices.

12.2 Implementing Secure Authentication

Developing a login system using SHA-256 hashed passwords stored in `users.json` enhanced our understanding of access control. We learned to secure user credentials and handle authentication errors, ensuring only authorized users (e.g., `Risers/12345678`) access the system.

12.3 Mastery of Network Programming

Using TCP sockets on `localhost:12345` improved our skills in client-server communication. We mastered socket setup, username exchange, and threading for concurrent message handling, ensuring efficient and stable network operations.

12.4 GUI Development and Usability

Building Tkinter interfaces with color-coded logs (green: senders, blue: system, red: errors), status indicators, and authentication dialogs taught us to prioritize user experience. Thread-safe updates via `master.after` ensured a responsive and intuitive GUI.

12.5 Data Persistence and File Handling

Implementing `chat_history.json` for persistent storage honed our skills in JSON serialization and file I/O. We optimized file operations to maintain data integrity during rapid updates, enabling seamless conversation continuity.

12.6 Error Handling and Robustness

The project emphasized robust error handling for network disruptions, authentication failures, and decryption errors. Comprehensive logging and graceful recovery strategies ensured system stability and user-friendly feedback.

12.7 Problem-Solving and Critical Thinking

Addressing challenges like authentication integration, thread synchronization, and GUI responsiveness sharpened our problem-solving skills. Iterative testing and refinement balanced security, functionality, and usability.

12.8 Software Engineering Practices

The modular structure (`crypto_utils.py`, `chat_server.py`, `chat_client.py`) and rigorous testing (unit, integration, usability) reinforced best practices. We learned to design maintainable systems and document processes effectively.

These outcomes equipped us with practical skills in secure system design, preparing us for advanced projects in communication and software development.

CHAPTER 13

PROJECT IMPACT

The secure chat application significantly enhances secure communication for personal messaging, small-scale professional collaboration, and educational purposes. By integrating AES-CBC encryption and SHA-256-based authentication, it ensures message confidentiality and restricts access to authorized users. The intuitive Tkinter GUI, color-coded message logs, and persistent chat history in `chat_history.json` make it accessible to non-technical users, while robust error handling guarantees reliability.

The project's impact includes:

- **Enhanced Privacy:** Encrypted messages and secure authentication protect sensitive data, building user trust in digital communication.
- **Educational Value:** The system demonstrates cryptography, authentication, networking, and GUI design, serving as a valuable learning tool for students and developers.
- **Operational Reliability:** Thread-safe operations and comprehensive error handling ensure stable performance in real-world scenarios.
- **Cost-Effectiveness:** Developed with open-source tools, the application is ideal for low-budget deployments in small organizations or academic settings.

The modular design and robust foundation make this application a reliable and secure communication solution, with strong potential for adoption in privacy-focused environments.

CHAPTER 14

CONCLUSION

The secure chat application successfully integrates AES-CBC encryption, SHA-256-based authentication, client-server networking, and a Tkinter-based GUI to deliver a highly secure and user-friendly communication platform. AES encryption with PKCS7 padding and random IVs ensures message confidentiality, while authentication (e.g., Risers/12345678) restricts access to authorized users via users.json. TCP sockets on localhost:12345 facilitate reliable real-time messaging, and the intuitive GUI provides clear controls, status indicators, and color-coded logs (green: senders, blue: system, red: errors) for enhanced usability. Chat history persistence in chat_history.json enables seamless conversation continuity, and robust error handling maintains system stability across various scenarios.

The project adeptly addressed challenges such as secure data transmission, user authentication, concurrency, and usability through a modular design (crypto_utils.py, chat_server.py, chat_client.py) and rigorous testing. Despite the limitation of a hardcoded encryption key, the system establishes a robust foundation for advanced security enhancements. The development process fostered deep expertise in cryptography, authentication, networking, GUI design, and software engineering, equipping the team for future secure system development.

By prioritizing robust security and user-centric design, this project contributes significantly to safe digital communication, with practical applications in education, small-scale collaboration, and privacy-focused environments. It exemplifies the effective synergy of encryption, authentication, and usability in modern communication systems.

CHAPTER 15

REFERENCES

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
 - Provides a comprehensive overview of cryptographic techniques, including AES-CBC and padding schemes, guiding the project's encryption module.
2. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
 - Covers TCP socket programming and client-server architectures, informing the networking implementation on localhost:12345.
3. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
 - A key resource for Python programming, including Tkinter for GUI development, threading for concurrency, and hashlib for SHA-256 authentication.
4. McKinney, W. (2017). *Python for Data Analysis* (2nd ed.). O'Reilly Media.
 - Discusses JSON handling and file I/O, supporting the implementation of chat_history.json and users.json persistence.
5. Python Software Foundation. (2025). *Python Documentation: socket, threading, tkinter, json, hashlib modules*. Retrieved from <https://docs.python.org/3/>.
 - Official documentation for Python libraries used in encryption, authentication, networking, and GUI components.
6. Schneier, B. (1996). *Applied Cryptography: Protocols, Algorithms, and Source Code in C* (2nd ed.). Wiley.
 - Offers practical insights into cryptography, influencing the design of the AES encryption and authentication mechanisms.